

Assignment 3

Due Wednesday, March 26

The assignment contains three parts. Part I (the common part for both 4MA433 and 4MA923) consists of a written part and an implementations part. Part II (only for 4MA433) is the theoretical part. It contains more theoretical problems sometimes depending on knowledge of abstract algebra. Part III (only for 4MA923) is the computational part. It contains more advanced programming problems.

- It is fine to work together as long as you write and understand your own solutions. (No copying!)
- To each assignment each student should submit three files on Moodle:
 1. Part I (written part) as a pdf-file. You can use L^AT_EX for this part, but it is also fine to submit scanned handwritten solutions. You are responsible for these being readable.
 2. Part I (implementation part) as a Python file (.py) or a Jupyter file(.ipynb)
 3. Students in 4MA433 submit Part II as a pdf-file. Students in 4MA923 submit Part III as a Python- or Jupyter-file.
- On the seminars the problems of the Part I is presented and discussed. Part II and Part III are discussed on the oral exam in the end of the course.
- Part I is graded according to a linear scale. To pass (grade E) 50% is needed, for grade C, 70% is needed and for grade A, 90%. Part II and Part III is graded together with the oral exam.

Part I (common part)

Implementation part

1. Let $E : y^2 = x^3 + Ax + B$ be an elliptic curve over $\mathbb{F}_p$ for some $p \geq 5$. We will use the pair $E=[A,B]$ for representing the curve. A point on $E(\mathbb{F}_p)$ can be either a pair $P = (x,y) \in \mathbb{F}_p^2$ or the point at infinity $\mathcal{O}$. We will represent P by $[x,y]$ and the point at infinity by the character '0'.

Let $p = 2^{256} - 2^{32} - 977$. In Bitcoin the curve $E : y^2 = x^3 + 7$ over $\mathbb{F}_p$ is used to construct a digital signature (of transactions). The point $G = (x,y)$ is used as the base for underlying discrete logarithm problem. In hexadecimal expansion (base 16) we have:

$$\begin{aligned} x &= 79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798 \\ y &= 483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8 \end{aligned}$$

Show that G is a point in $E(\mathbb{F}_p)$ by using the function `pointOnCurve`. *Hint:* If you prepend 0x to an hexadecimal expression you can easily convert it to an integer in base 10 by `int(expression)`.

2. Let $E : y^2 = x^3 + 5x + 12$. Generate $E(\mathbb{F}_{13})$ by using `generateCurve`.
3. (a) Write the function `addPoints(P,Q,E,p)`. It should return the sum $P \oplus Q$. Remember that both P , Q or $P \oplus Q$ can be $\mathcal{O}$.

- (b) Try it out by computing sums of points on the curve $E : y^2 = x^3 + 5x + 12$ over $\mathbb{F}_{13}$. Compute $\mathcal{O} \oplus \mathcal{O}$, $\mathcal{O} \oplus P$ and $Q \oplus \mathcal{O}$ for some $P \neq \mathcal{O}$ and $Q \neq \mathcal{O}$ and at least five more sums.
4. Now we will mimic the fast powering algorithm for producing the double-and-add algorithm.
- (a) Write the function `doubleAndAdd(P,n,E,p)` for computing the point nP in $E(\mathbb{F}_p)$. Try to mimic the `fast2Power` function we have used.
- (b) Back to the Bitcoin curve in problem 1. Compute nG for $n = 165717357988647532$.
5. Implement Lenstra's elliptic curve factorizing method.
- (a) Write a function `factorLenstra(N)` that tries to find a non-trivial factor of N . You will need your `addPoints`- and `doubleAndAdd`- functions, but you will need to change them so that they output reasonable information if they fail to add points on the elliptic curve.
- (b) Try to find a non-trivial factor of the following numbers by using the `factorLenstra`-function

$$N_1 = 516083$$

$$N_2 = 1833779393$$

$$N_3 = 13487843290022369713$$

6. In this problem we will implement the Elliptic Elgamal cryptosystem over $E(\mathbb{F}_p)$. We assume that $p \equiv 3 \pmod{4}$. You can use the `messageToPoint`-functions below to encode the message. You assume that we have a point P on the curve of large order.
- (a) Write the function `elgamalEllipticEncrypt(P,QA,E,p,m)` that encrypts the message m (integer) using the public key Q_A . You should use the `messageToPoint` - function. You should also handle the message expansion problem and not send both the coordinates of the ciphertext points. See also Problem 7.
- (b) Write the function `elgamalEllipticDecrypt(C,P,nA,E,p,K=100)` where C is the ciphertext in the form of the output from `elgamalEllipticEncrypt`. The output of the function should be the original message m .

7. The Bitcoin curve from is used for an Elliptic Elgamal crytosystem. Let $P = (x, y)$ where

$$x = 79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798$$

$$y = 483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8$$

in hexadecimal expansion. We also know that the order of P is

$$FFFEBAAEDCE6AF48A03BBFD25E8CD0364141$$

in hexadecimal expansion. The private key is

$$n_A = 39848972070998439288129330129747770457517229308860377048978242893896467667939$$

The ciphertext is

$$x(C_1) = 14597780031786705375271868769108467138015647678572751776302579165318028652678$$

$$b_1 = 1$$

$$x(C_2) = 21210963020721916651969697565057900349212558568934755775237286996818078542813$$

$$b_2 = 0$$

I have used an extra bit to denote the "sign" of the y -value of the ciphertext points. b_1 gives the sign of $y(C_1)$ and b_2 the sign of $y(C_2)$ ($b_1 = 1$ means that $y(C_1) > (p-1)/2$ and $b_2 = 0$ means that $y(C_2) \leq (p-1)/2$). Find the plaintext! The error tolerance $K = 100$ is used.

8. (a) Implement Gauss reduction algorithm by writing a function `reductionGauss(v1,v2)`. The input is two linear independent vectors in $\mathbb{Z}^2$ and the output should be a shortest vector in the lattice generated by v_1 and v_2 .
 (b) Find a smallest vector for the lattice generated by $v_1 = (96, 58)$, $v_2 = (101, 61)$.
 (c) Find a smallest vector for the lattice generated by $v_1 = (66586820, 65354729)$ and $v_2 = (6513996, 6393464)$.
9. We are going to implement the NTRU cryptosystem. We need primes N , p and q . We also need a parameter d such that $q > (6d+1)p$. We can choose $N \approx 3d$.
 (a) Write a function `generateTrinary(d1,d2,N)` that produces a random list of length N , containing d_1 elements equal to 1, d_2 elements equal to -1 and the rest of the elements equal to 0.
 (b) Write a function `generateNTRU(d,p,q,N)` that generates the private keys f and g (by using the `generateTrinary` function) and computes the inverses f_p^{-1} and f_q^{-1} of f modulo p and modulo q respectively. Compute also the public key h . The output from the function should be h , f and f_p^{-1} .
 (c) Write the functions `textToPoly(s,p,N)` and `polyToText(coeff_list,p,N)` that transforms a text string s to a list of coefficients modulo p (of a polynomial of degree at most N) and vice versa. You can for example use your old functions `textToInt` and `intToText`. The function `getExpansion` is also useful.
 (d) Write a function `encryptNTRU(d,p,q,N,m,h)`. The plaintext m should be a list of integers modulo p . NOTE: Before you use m in the calculation of the ciphertext e you need to center lift it to a list of integers. The output from the function should be the ciphertext e (either as a polynomial or as a list).
 (e) Write a function `decryptNTRU(d,p,q,N,f,f_p_inv,e)`. The output should be a list of coefficients representing the message. Before you output the list it is a good idea to first convert it to a list of integers (otherwise it might be hard for the `polyToText` function to read it).
 (f) Let us try our NTRU system. Let $d = 12$, $p = 257$, $q = 18773$ and $N = 31$. Generate the public and private keys. Encrypt a message of consisting of maximum 30 characters. Decrypt the ciphertext you get. Did you recover the message?

Written part

10. Consider the curve $E : y^2 = x^3 + x + 3$ over $\mathbb{F}_7$.

- (a) Compute the addition table for this curve.
- (b) Show that $G = (4, 1)$ is a generator for the group $E(\mathbb{F}_7)$.
- (c) Find points of order 1, 2 and 3.

11. Consider $E : y^2 = x^3 + 23x + 13$ over $\mathbb{F}_{83}$. Let $P = (3, 21)$ be a point on the curve. Compute $19P$ using the double-and-add algorithm.
12. Let L be the lattice with basis $v_1, v_2, \dots, v_n$ in $\mathbb{R}^m$, where $m > n$. In this problem we will prove that $\det(L)$ (the volume of the fundamental domain) is given by $\sqrt{\det(B^T \cdot B)}$, where B is the matrix consisting of the basis vectors as columns.
- (a) Let U be the n -dimensional subspace in $\mathbb{R}^m$ generated by $v_1, v_2, \dots, v_n$, hence $U = \text{span}(v_1, v_2, \dots, v_n)$. Explain why we can find an orthonormal basis $f_1, f_2, \dots, f_n$ of U and an $n \times n$ matrix $A = (a_{ij})$ such that

$$\begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix} \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix}.$$

- (b) Prove that

$$\begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix} (f_1 f_2 \dots f_n) = I_n,$$

where I_n is the $n \times n$ identity matrix.

- (c) Show that

$$B^T \cdot B = A \cdot A^T$$

and that

$$\det(B^T \cdot B) = \det(A)^2.$$

- (d) Motivate why $\det(L) = \det(A)$ and conclude that

$$\det(L) = \sqrt{\det(B^T \cdot B)}.$$

Part II (Theoretical part)

Only for students in 4MA433.

1. Let $E(\mathbb{F}_p)$ be the additive group of order n on an elliptic curve with the binary operation $\oplus$ defined in the way we did in class. Assume also that there is a point G on the curve that generates the whole curve. That is,

$$\{kG \mid k = 1, 2, \dots, n\} = E(\mathbb{F}_p).$$

We define the order of an element $H \in E(\mathbb{F}_p)$ as the smallest positive integer m such that $mH = \mathcal{O}$. We denote this $\text{ord}(H) = m$. We also have $\text{ord}(G) = n$.

- (a) Let $s \in \mathbb{Z}^+$ such that $sH = \mathcal{O}$. Show that $\text{ord}(H) \mid s$. *Hint:* Use the division algorithm.
- (b) Prove that $\text{ord}(kG) = \frac{n}{\gcd(n, k)}$. *Hint:* Study $\text{ord}(H)H$ to conclude that $\frac{n}{\gcd(n, k)} \mid \text{ord}(H)$ and study $\frac{n}{\gcd(n, k)}H$ to conclude that $\text{ord}(H) \mid \frac{n}{\gcd(n, k)}$.

2. Let us study the Elliptic Curve DSA algorithm in detail.

- (a) Assume that you get a signature (s_1, s_2) of a document d where $s_1 = 0$. Why should you be suspicious?
- (b) Let us assume that Samantha has signed two different documents by using the same random number e in the signing algorithm. Show that it is possible to compute Samantha's secret signing key s (assuming $s_1 \neq 0$).

3. We will show that the basis $v_1, v_2, \dots, v_n$ given by the LLL algorithm is size reduced. That is, we need to show that

$$|\mu_{k,j}| = \frac{|v_k \cdot v_j^*|}{\|v_j^*\|^2} \leq \frac{1}{2}.$$

for all $2 \leq k \leq n$ and $1 \leq j \leq k-1$.

- (a) Show it for the case $n = 2$.

For the general case let us assume that the vectors $v_1, \dots, v_{k-1}$ are all size reduced. This means that $|\mu_{i,j}| \leq 1/2$ for $2 \leq i \leq k-1$ and $1 \leq j \leq i-1$. We will now prove that the new vector $\tilde{v}_k$ given after the k round in the outer loop is also size reduced.

- (b) If we replace v_k by $\tilde{v}_k = v_k - cv_j$ (where $j < k$ and $c \in \mathbb{Z}$) the associated Gram-Schmidt basis does not change. Prove this! *Hint:* Prove that $\tilde{v}_k^* = v_k^*$.
- (c) Let $\tilde{v}_k = v_k - \lfloor \mu_{k,k-1} \rfloor v_{k-1}$, where $\mu_{k,k-1} = \frac{v_k \cdot v_{k-1}^*}{\|v_{k-1}^*\|^2}$. This corresponds to the first step in the size reduction (inner) loop. We can also express the new vector in the associated Gram-Schmidt basis (which are the same according to last problem)

$$\tilde{v}_k = v_k^* + \sum_{i=1}^{k-1} \tilde{\mu}_{k,i} v_i^*$$

where $\tilde{\mu}_{k,i} = \frac{\tilde{v}_k \cdot v_i^*}{\|v_i^*\|^2}$. Show that $|\tilde{\mu}_{k,k-1}| \leq 1/2$. *Hint:* Express also v_k and v_{k-1} in the GS-basis.

- (d) Do the same for $\tilde{v}_k = v_k - \lfloor \mu_{k,k-2} \rfloor v_{k-2}$ (where v_k now is $\tilde{v}_k$ from last problem) and prove that $|\tilde{\mu}_{k,k-2}| \leq 1/2$
- (e) If we continue in the similar way we can prove that $|\tilde{\mu}_{k,j}| \leq 1/2$ for all $1 \leq j \leq k-1$. Why is it important to go from $j = k-1$ to $j = 1$ and not in the opposite direction?

4. We will now look closer on Lovasz condition in the LLL algorithm:

$$\|v_k^*\|^2 \geq \left(\frac{3}{4} - \mu_{k,k-1}^2 \right) \|v_{k-1}^*\|^2.$$

- (a) Show that it is equivalent to

$$\frac{3}{4} \|v_{k-1}^*\|^2 \leq \|\mu_{k,k-1} v_{k-1}^* + v_k^*\|^2.$$

- (b) Let U_{k-2} be the orthogonal complement to $\text{span}(v_1, v_2, \dots, v_{k-2})$, hence we have

$$U_{k-2} = \{u \in \mathbb{R}^n; u \cdot v = 0 \text{ for all } v \in \text{span}(v_1, v_2, \dots, v_{k-2})\}.$$

Let $P_{k-2}(v)$ be the orthogonal projection of v onto U_{k-2} .

Motivate why we also can describe Lovasz condition as

$$\frac{3}{4} \|P_{k-2}(v_{k-1})\|^2 \leq \|P_{k-2}(v_k)\|^2.$$

Part III (Computational part)

Only for students in 4MA923.

1. Implement the Elliptic curve version of DSA.

- (a) Write a function `generateEllipticDSAKey(P,q,E,p)` where E is an elliptic curve over $\mathbb{F}_p$ and P is a point on $E(\mathbb{F}_p)$ of order q . The output should be the private signing key s and the public verification key V .
- (b) Write the signing function `signEllipticDSA(d,s,P,q,E,p)`, where d is the document (modulo q) and s is the signing key. The output should be the signature (s_1, s_2) .
- (c) Write the verification function `verifyEllipticDSA(d,sign,V,P,q,E,p)`, where d is the document and V is the public verification key.

2. Now you should use your functions from last problem. Let us consider the so called Bitcoin curve $E : y^2 = x^3 + 7$ over $\mathbb{F}_p$ where $p = 2^{256} - 2^{32} - 977$. Let $P = (x, y)$ where

$$\begin{aligned} x &= 79BE667EF9DCBBAC55A06295CE870B07029BFCDDB2DCE28D959F2815B16F81798 \\ y &= 483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8 \end{aligned}$$

in hexadecimal expansion. We also know that the order of P is

$$\text{FFFEBAAEDCE6AF48A03BBFD25E8CD0364141}$$

in hexadecimal expansion. Now it is time to generate an Elliptic DSA system by using this curve.

- (a) Generate a private key and public verification key
- (b) Sign the document "This is the right document" where you use your private key. Then verify that your signature was correct.
- (c) I have the public verification key

$$V = (7291208038029293503904516696354384727804822870539627941234890182717184024858, 113153368912782601077844337646024970272543479067460718629145014764499291840063)$$

Which of the messages

$$\begin{aligned} d_1 &= 67808320894793080942835777731302085498712874166089 \\ d_2 &= 4443886118161159352654130030329805709934681456609994569 \end{aligned}$$

have I sign by the signature (s_1, s_2) where

$s_1 = 5827829480990925927172858220912447702661270096536761017834280182895861321067,$

$s_2 = 8435058229815304398357493929165388053892875055370562927944208609314415236314?$

What was the contents of the right message?

3. In this problem we will implement the LLL algorithm and test it.

- (a) Write a function `LLL(B,delta=3/4)` where B is a list of vectors spanning a lattice L . You need also to implement a `gramSchmidt` function that gives you the Gram Schmidt basis. The LLL algorithm will work for values of `delta` between $1/4$ and 1 (excluding these values).
- (b) Let B be a list of n vectors v_i in $\mathbb{Z}^n$. Write a function `hadamardRatio(B)` that computes the Hadamard Ratio

$$\mathcal{H}(B) = \left(\frac{\det(B)}{\prod_{i=1}^n \|v_i\|} \right)^{1/n}.$$

- (c) Let

$$B = \begin{pmatrix} 19 & 2 & 32 & 46 & 3 & 33 \\ 15 & 42 & 11 & 0 & 3 & 24 \\ 43 & 15 & 0 & 24 & 4 & 16 \\ 20 & 44 & 44 & 0 & 18 & 15 \\ 0 & 48 & 35 & 16 & 31 & 31 \\ 48 & 33 & 32 & 9 & 1 & 29 \end{pmatrix}$$

where we let the rows be the basis for a lattice in $\mathbb{R}^6$. Compute the Hadamard ratio for this basis. Use the LLL algorithm to find an LLL-reduced basis. Compute the Hadamard ratio also for this basis. Compare the results. Also compare your results to the example in HPS.

- 4. Alice and Bob are using the NTRU system with parameters $d = 2$, $p = 3$, $q = 59$ and $N = 11$. The public key is

$$h(x) = 37 + 20x + 11x^2 + 6x^3 + 34x^4 + 52x^5 + 28x^6 + 46x^7 + 4x^8 + 13x^9 + 44x^{10}$$

Eve has found the ciphertext

$$e(x) = 33 + 8x + 32x^2 + 26x^3 + 26x^4 + 47x^5 + 21x^6 + 40x^7 + 32x^8 + 0 \cdot x^9 + 33x^{10}$$

In this problem you will help Eve to find the plaintext.¹

- (a) Generate the matrix for the corresponding NTRU-lattice. Then run the LLL algorithm on this matrix (where the rows are the basis for the lattice).
- (b) Solve the NTRU key recovery problem. Do you know if you have found the right keys or rotations of them? Does it matter when it comes to decryption?
- (c) Use the found key(s) to find the plaintext. *Hint:* The plaintext consists of two lowercase letters.

¹We (again) ignore the ethical consequences of helping Eve.

Some Algorithms

From last Assignment we have the following algorithm for finding square roots of an integer modulo a prime.

```

1 def findSquareRoot(N,p):
2     N0=N%p
3     if fast2Power(N0,(p-1)//2,p)==1: #Euler's criterion
4         if p%4==3:
5             x1=fast2Power(N0,(p+1)//4,p)
6             y1=p-x1
7             return [x1,y1]
8         else:
9             for i in range(1,((p-1)//2)+1):
10                 if (i*i)%p==N0:
11                     x1=i
12                     y1=p-i
13                     return [x1,y1]
14     return []

```

Here follows some functions related to elliptic curves

```

1 #We represent Elliptic curves  $E:y^2=x^3+Ax+B$  by the pair of integers [A,B]
2 def isElliptic(E,p):
3     A=E[0]
4     B=E[1]
5     discr=(4*(A**3)+27*(B**2))%p
6     return discr!=0
7
8 def pointOnCurve(P,E,p):
9     if P=='0':
10         return True
11     else:
12         A=E[0]
13         B=E[1]
14         x=P[0]
15         y=P[1]
16         return (y**2)%p==(x**3+A*x+B)%p
17
18 def generateCurve(E,p):
19     if isElliptic(E,p)==False:
20         print("This is not an elliptic curve")
21         return None
22     A,B = E
23     listOfPoints=['0']
24     for x in range(p):
25         a=(x**3+A*x+B)%p
26         if a==0:
27             listOfPoints.append([x,0])
28         if fast2Power(a,(p-1)//2,p)==1: #Euler's criterion there are solutions
29             y1,y2=findSquareRoot(a,p)
30             listOfPoints.append([x,y1])
31             listOfPoints.append([x,y2])
32     return listOfPoints
33
34
35 def orderCurve(E,p):
36     if isElliptic(E,p)==False:

```



```
37     print("This is not an elliptic curve")
38     return None
39 A,B = E
40 numberOfPoints=1 #'0' is always a point on the curve
41 for x in range(p):
42     a=(x**3+A*x+B)%p
43     if a==0:
44         numberOfPoints=numberOfPoints+1
45     if fast2Power(a,(p-1)//2,p)==1: #Euler's criterion there are solutions
46         numberOfPoints=numberOfPoints+2
47 return numberOfPoints
48
49 #How to encode a message to a point
50 def messageToPoint(m,E,p,K=100):
51     #K is the error tollerance
52     #Message m needs to satisfy (m+1)L<p
53     if (m+1)*K>= p:
54         print("Error tolerance or size of message space need to be changed!")
55         return None
56     for j in range(0,K-1):
57         x=K*m+j
58         z=(x**3+E[0]*x+E[1])%p
59         #print(x,z)
60         if fast2Power(z,(p-1)//2,p)==1:
61             y=fast2Power(z,(p+1)//4,p)
62             M=[x,y]
63             #print(P)
64             return M
65 print("No encoding found, increase K")
66 return None
```